# The Reel Deal: Movie Ticket Booking System

## System Requirements Documentation

Drew Busz

Luddy School of Informatics, Computing, and Engineering, Indiana University Indianapolis

INFO-C450: System Design

Professor Jafrina Jabin

May 4, 2026

# Table of Contents


Customer Problem Statements & System Requirements ..... 3

Customer Problem Statement: ..... 3

Glossary of Terms: ..... 3

System Requirements ..... 4

Functional Requirements: ..... 4

Nonfunctional Requirements (FURPS): ..... 4

User Interface Requirements: ..... 5

Functional Requirement Specification ..... 7

Use Cases ..... 8

Use Case Diagram ..... 9

Class Diagram ..... 9

System Sequence Diagram and Activity Diagram ..... 10

System Sequence Diagrams: ..... 10

Activity Diagrams: ..... 13

User Interface Specification ..... 15

User Effort Estimation: ..... 18

Traceability Matrix ..... 19

The Reel Deal Movie Booking System ..... 20

System Architecture and System Design ..... 20

UML Package Diagram: ..... 21

Global Control Flow ..... 22

Hardware Requirements ..... 23

User Interface Design and Implementation, Design of Tests ..... 23

Project Plan ..... 25

References ..... 27

# Customer Problem Statements & System Requirements

**The Reel Deal: Movie Ticket Booking System**

## Customer Problem Statement:

When I go to a movie theater, I usually have no idea if there are still good seats left for the showtime I want. Even if I get there early, I may still end up waiting in line, only to find that the remaining seats are scattered or already taken. If I go with friends or family, it can be frustrating trying to find seats together. This often wastes time and takes away from the experience before the movie even starts.

Theater staff deal with similar problems. They have to manage movie listings, set up showtimes, keep track of which seats are available, and handle ticket sales all at once. Without a clear system to organize this information, it can lead to delays, confusion, and seat assignment mistakes.

As a customer, I would expect a system where I can see movies and showtimes ahead of time, check which seats are open, reserve seats before arriving, and receive confirmation of my booking. Theater staff would expect a system that helps them manage movies, showtimes, and seating in a simple and organized way.

## Glossary of Terms:

| Term | Definition |
|---|---|
| Movie | A film available for customers to book in the system |
| Showtime | A scheduled time when a movie plays in a specific auditorium |
| Auditorium | A theater room with a defined seating layout |
| Seat | A selectable position in an auditorium |
| Booking | A confirmed reservation of one or more seats for a showtime |
| Ticket | Proof of a completed booking |
| Manager | User role that manages movies, showtimes, and seating |
| Staff | User role that validates tickets and assists customers |

# System Requirements

## Functional Requirements:

| REQ # | Priority Weight | Description |
|---|---|---|
| REQ-1 | High | The system should allow staff to add new movies with descriptions and poster images. |
| REQ-2 | High | The system should allow managers to create and manage showtimes |
| REQ-3 | High | The system should automatically create a standard A-D by 1-8 seating layout for each new showtime. |
| REQ-4 | High | The system should display seat availability for each showtime |
| REQ-5 | High | Customers should be able to select and reserve available seats |
| REQ-6 | High | The system should prevent double booking of the same seat |
| REQ-7 | Medium | Staff should be able to view booking records through a staff booking dashboard. |
| REQ-8 | Medium | The system should allow customers to select multiple seats in one booking. |
| REQ-9 | High | The system should update seat status immediately after booking |
| REQ-10 | Medium | The system should allow users to create accounts and log in |
| REQ-11 | Low | The system should display movie details and descriptions |
| REQ-12 | High | Theater staff should be able to validate tickets for customers |

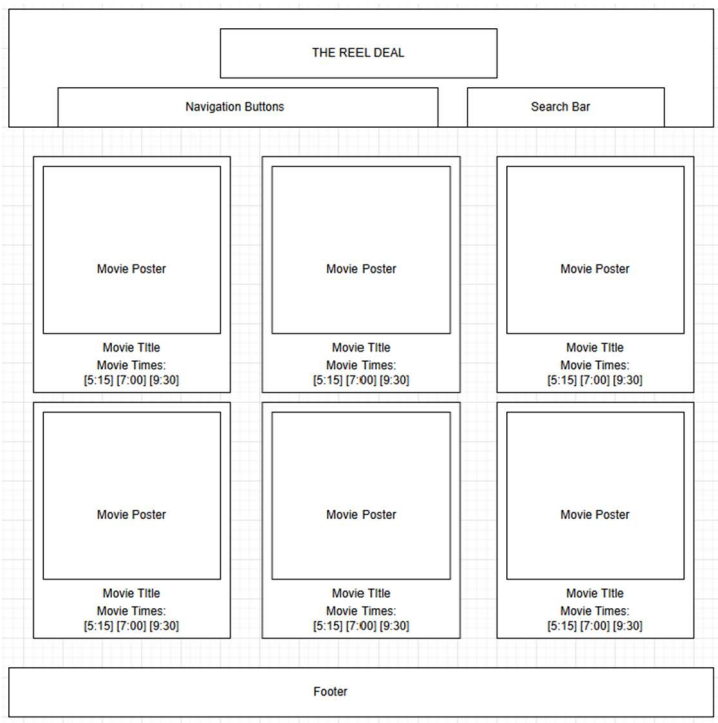
## Nonfunctional Requirements (FURPS):

| NFR # | Priority Weight | Description |
|---|---|---|
| NFR-1 | High | Seat availability should update within 2 seconds after a booking (Performance) |
| NFR-2 | High | The system should prevent data loss during booking transactions (Reliability) |
| NFR-3 | Medium | The system should be easy for first-time users to navigate (Usability) |
| NFR-4 | Medium | The system should be accessible from standard web browsers (Supportability) |
| NFR-5 | Medium | The interface should clearly show seats and availability (User Interface) |

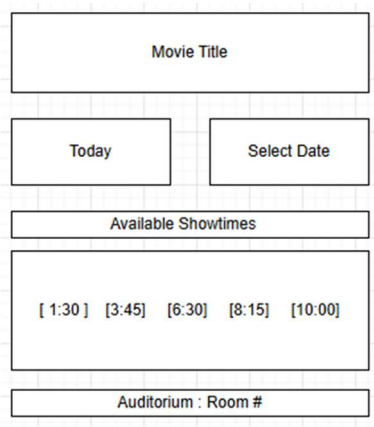
## User Interface Requirements:

The following user interface requirements describe the primary screens that users interact with when using the movie booking system.

UI-1: Movie List Page
The system should display all available movies and their showtimes.

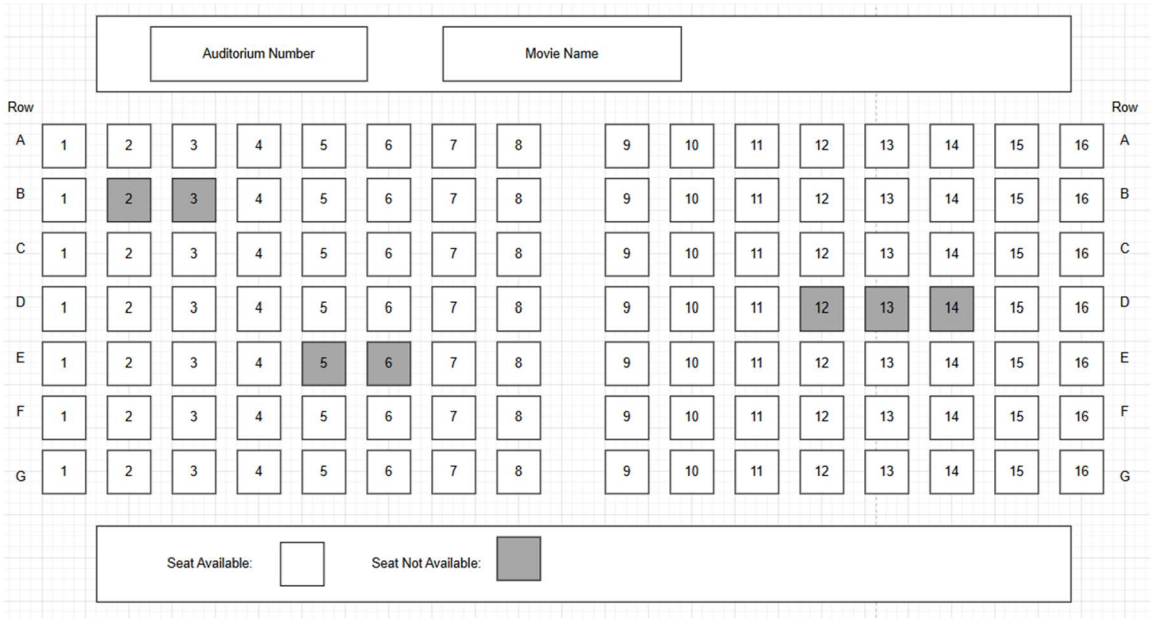



UI-2: Showtime Selection Page
The system should display available showtimes for a selected movie.



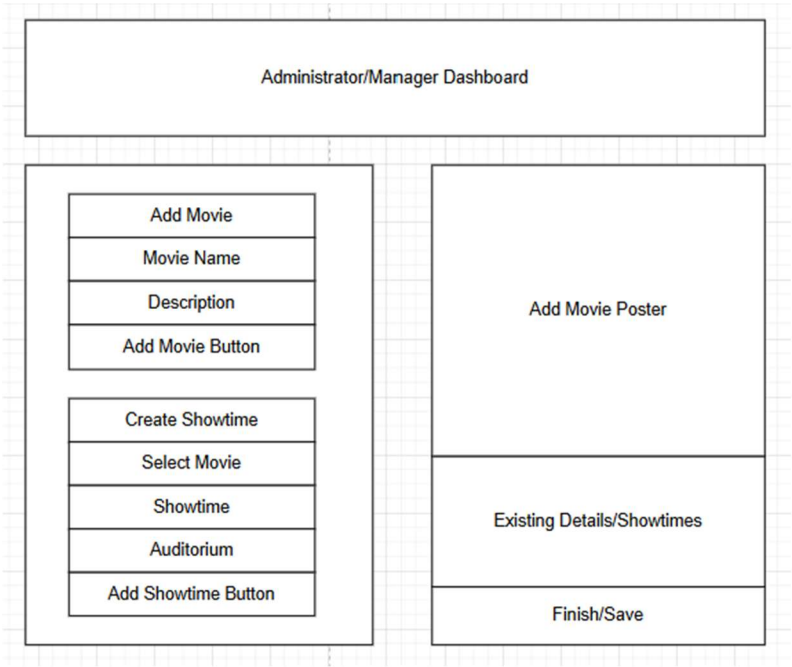
UI-3: Seat Selection Page

The system should display a seating layout with available and reserved seats clearly marked.




UI-4: Manager Control Page

The system should allow managers to add movies, create showtimes, and manage seating layouts.



UI-5: Booking Confirmation Page
The system should display booking confirmation details after a successful seat reservation.

# Functional Requirement Specification

**Stakeholders**

- Theater owner who wants an organized and efficient ticket booking system
- Theater manager responsible for managing movies, showtimes, and seating layouts
- Theater staff who validate tickets and assist customers
- Customers who want to reserve seats and purchase tickets in advance
- System developer responsible for building and maintaining the system

**Primary Actors**

- Customer: This actor can browse movies, select showtimes, view available seats, reserve seats, and receive a ticket confirmation.
- Theater Staff: This actor can validate customer tickets and view booking information to assist customers.
- Theater Manager: This actor can add, edit, and remove movies, create showtimes, and manage seating layouts.

**Secondary Actors**

- System: This actor is responsible for updating seat availability, storing booking records, and preventing double booking of seats.

## Use Cases

**Theater Manager (total: 16)**

- Add movie: To add a new movie to the system (3)
- Edit movie: To update movie details (2)
- Create showtime: To schedule a new showtime for a movie (3)
- Edit showtime: To update or remove a showtime (2)
- Define seating layout: To create and manage the seating layout of an auditorium (3)
- Login/Logout: To log in and out of the manager account (3)

**Customer (total: 14)**

- View movies: To see available movies and showtimes (2)
- Select showtime: To choose a showtime for a movie (2)
- View seats: To see which seats are available or reserved (3)
- Reserve seat: To select and reserve available seats (3)
- Confirm booking: To finalize the booking and receive ticket confirmation (2)
- View booking history: To view previous bookings (2)

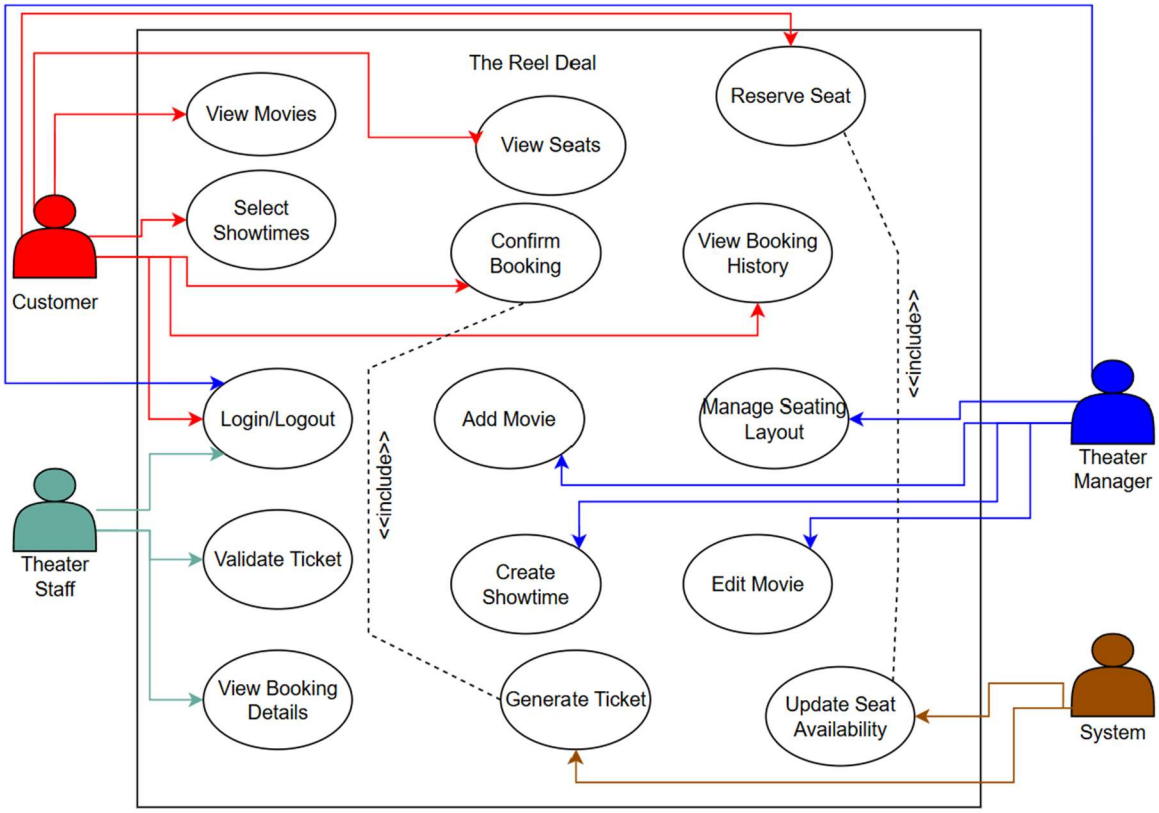
**Theater Staff (total: 6)**

- Login/Logout: To log in and out of the staff account (2)
- Validate ticket: To confirm the validity of a customer's ticket (2)
- View booking information: To check booking details when assisting customers (2)

**System (total: 8)**

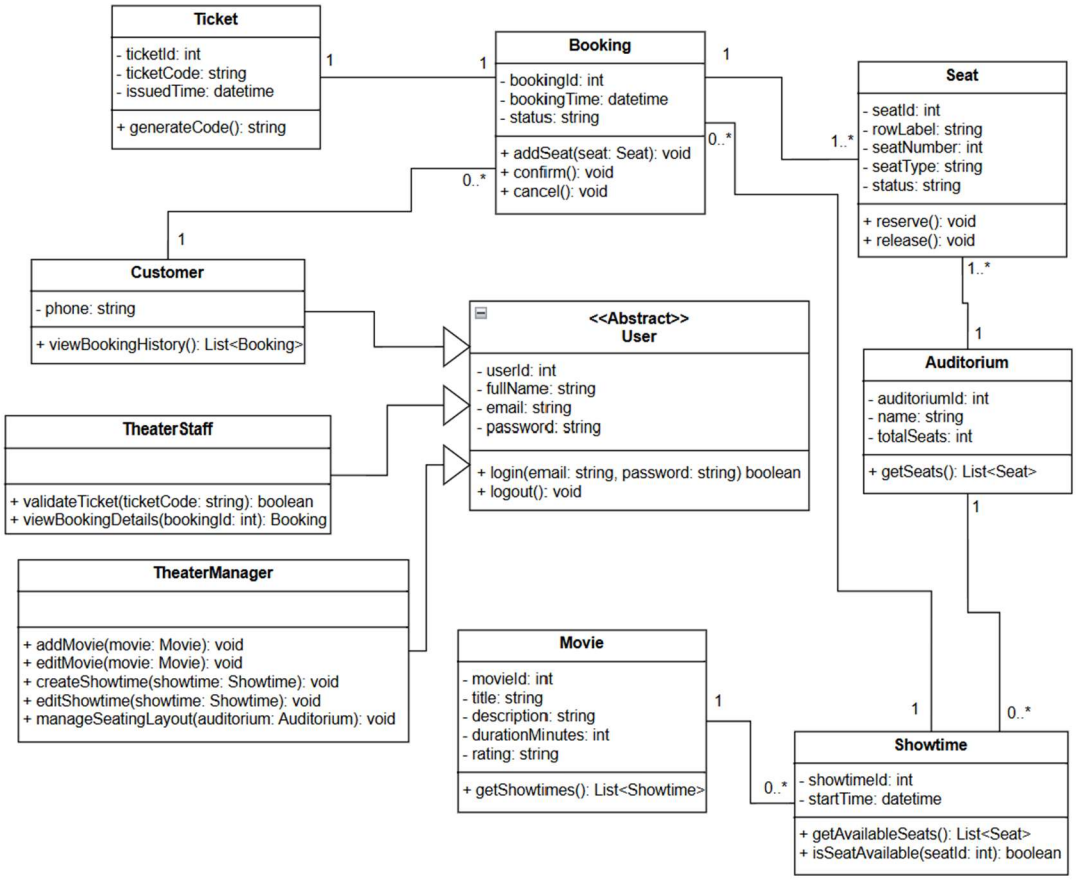
- Update seat availability: To automatically update seat status after booking (3)
- Prevent double booking: To ensure a seat cannot be reserved by more than one customer (3)
- Store booking records: To save booking and ticket data in the database (2)

## Use Case Diagram




## Class Diagram

The class diagram models movies, showtimes, auditoriums, seats, bookings, and tickets. Users inherit from an abstract User class to represent customer, staff, and manager roles. Multiplicity shows how one movie can have many showtimes, and one booking can include multiple seats. This diagram also enforces the rule that a seat can only be reserved once per showtime through the relationship between Booking, Seat, and Showtime.




# System Sequence Diagram and Activity Diagram

## System Sequence Diagrams:

**Sequence Diagram for Confirm Booking**

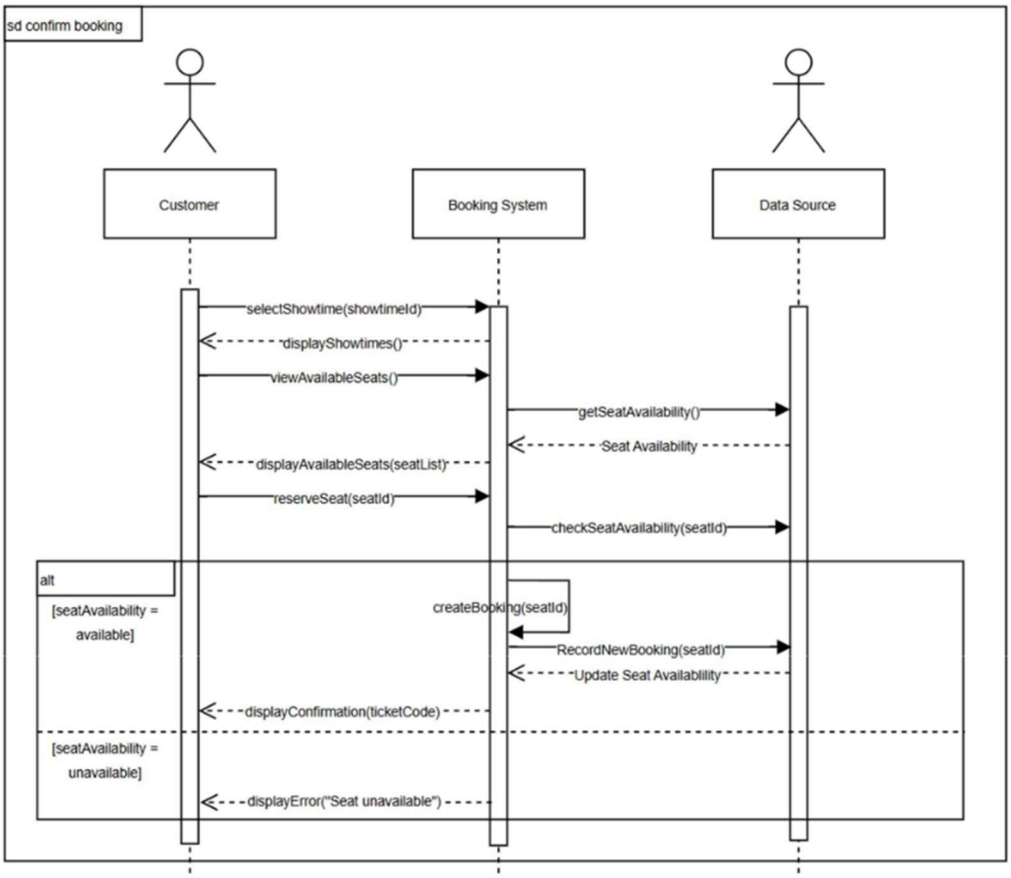
Actor: Customer
Object: Booking System
Interacts with the Data Source or Database

Steps of confirming a booking:

1. The Customer selects a movie and showtime.
2. The Customer requests to view available seats for the selected showtime.
3. The Booking System retrieves seat availability information from the data source or database.

4. The Booking System displays available seats to the Customer.
5. The Customer selects one or more seats and requests to reserve them.
6. The Booking System checks seat availability in the data source or database.
7. If the seats are available:
    1) A new booking record is created in the data source or database.
    2) The selected seats are marked as reserved in the data source or database.
    3) A ticket is generated and stored in the data source or database.
    4) A booking confirmation screen with ticket details is shown to the Customer.
8. If one or more seats are not available:
    1) The Booking System displays an error message indicating the seat is unavailable.
    2) The Customer is prompted to select different seats.

The sequence diagram is shown below:



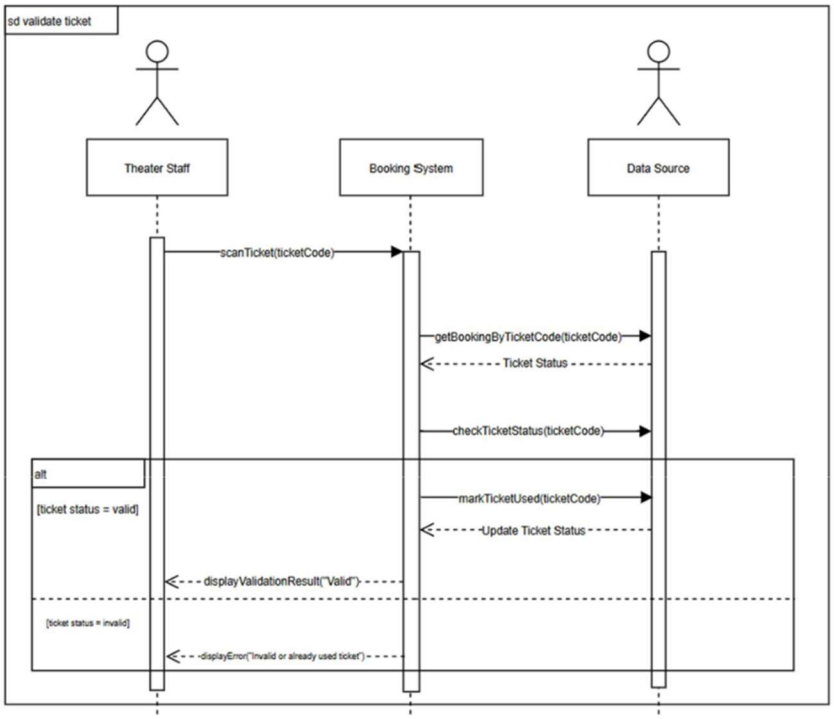
**Sequence Diagram for Validate Ticket**

Actor: Theater Staff
Object: Booking System
Interacts with the Data Source or Database

Steps of validating a ticket:

1. The Theater Staff scans or enters the ticket code.
2. The Theater Staff submits the ticket code to the Booking System for validation.
3. The Booking System retrieves the booking record associated with the ticket code from the data source or database.
4. The Booking System checks the ticket and booking status (for example: exists, not expired, not already used).
5. If the ticket is valid:
   1) The Booking System updates the ticket status to “used” in the data source or database.
   2) The Booking System displays a “Valid Ticket” result to the Theater Staff (entry allowed).
6. If the ticket is invalid (not found, already used, or canceled):
   1) The Booking System displays an error message to the Theater Staff (entry denied).

The sequence diagram is shown below:



## Activity Diagrams:

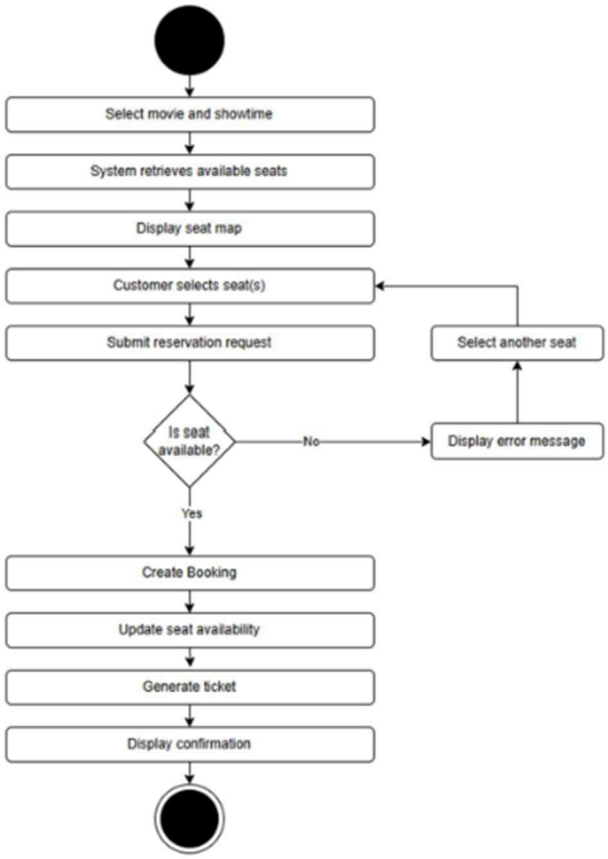
**Confirm Booking**

**States**

- Initial State: The customer selects a movie and showtime to book tickets.
- Final State:
    1. The customer receives a booking confirmation with a ticket code.
    2. The selected seat is unavailable, and the customer receives an error message and must choose another seat.

**Actions**

The customer selects a showtime and requests to view available seats. The system retrieves seat availability and displays the seat map. The customer selects a seat and submits a reservation request. The system checks whether the seat is still available. If it is available, the system creates a booking, updates seat availability, generates a ticket code, and displays the confirmation. If the seat is not available, the system displays an error message and prompts the customer to select a different seat.



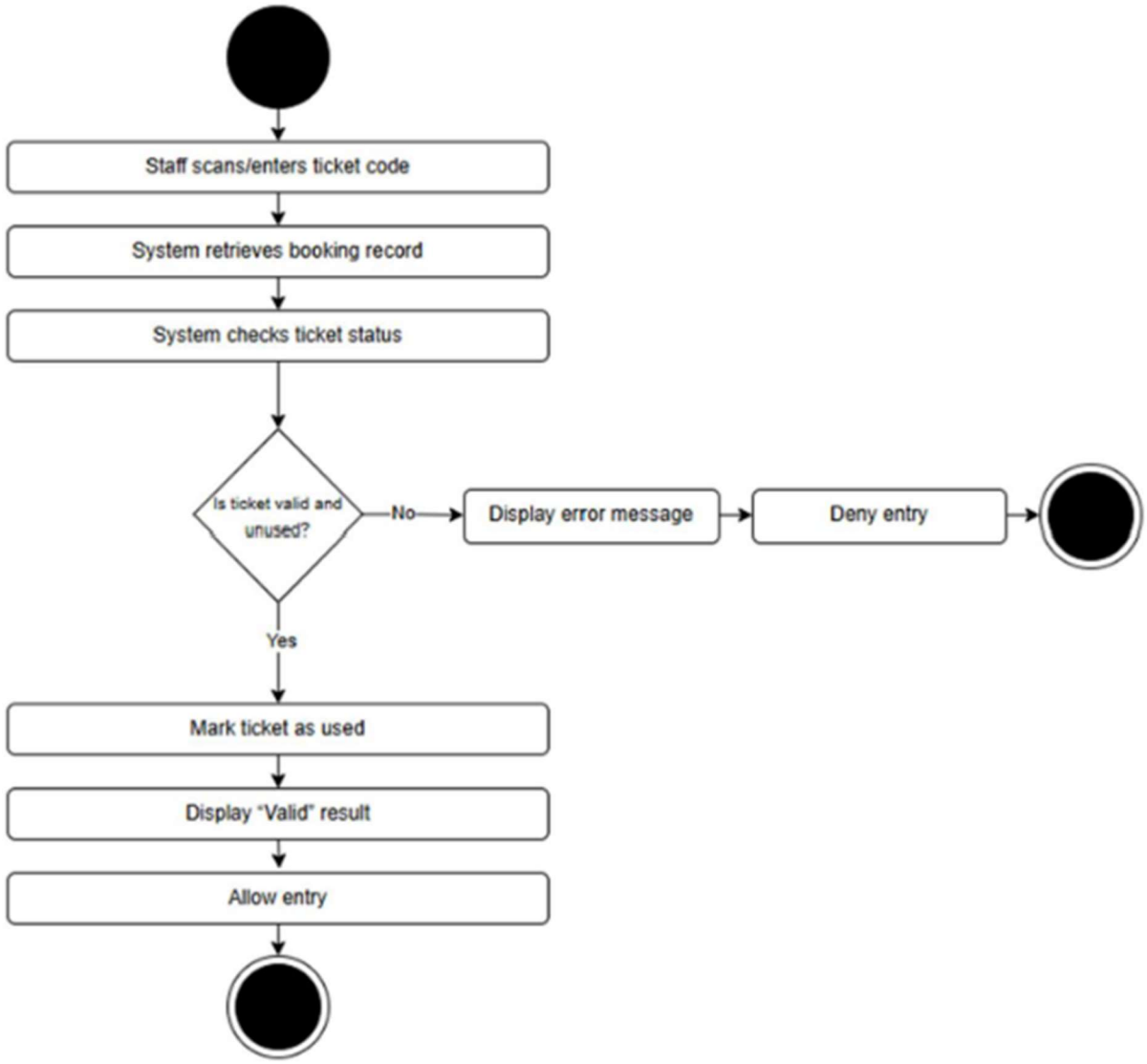
**Validate Ticket**

**States**

- Initial State: Theater staff scans or enters a ticket code to validate entry.
- Final State:
    1. The ticket is valid, and the system approves entry.
    2. The ticket is invalid (not found, canceled, or already used), and the system denies entry.

**Actions**

Theater staff scans or enters a ticket code at the theater entrance. The system retrieves the booking information associated with the ticket code and checks the ticket status. If the ticket is valid and unused, the system marks the ticket as used and displays a valid result to staff. If the ticket is invalid or already used, the system displays an error message and entry is denied.



# User Interface Specification

This document outlines the initial user interface design for The Reel Deal movie booking system. It shows how users will interact with the system and what each screen will display. The mockups are included to give a clear idea of how users navigate the system, enter information, and receive feedback after completing actions like selecting seats or confirming a booking.

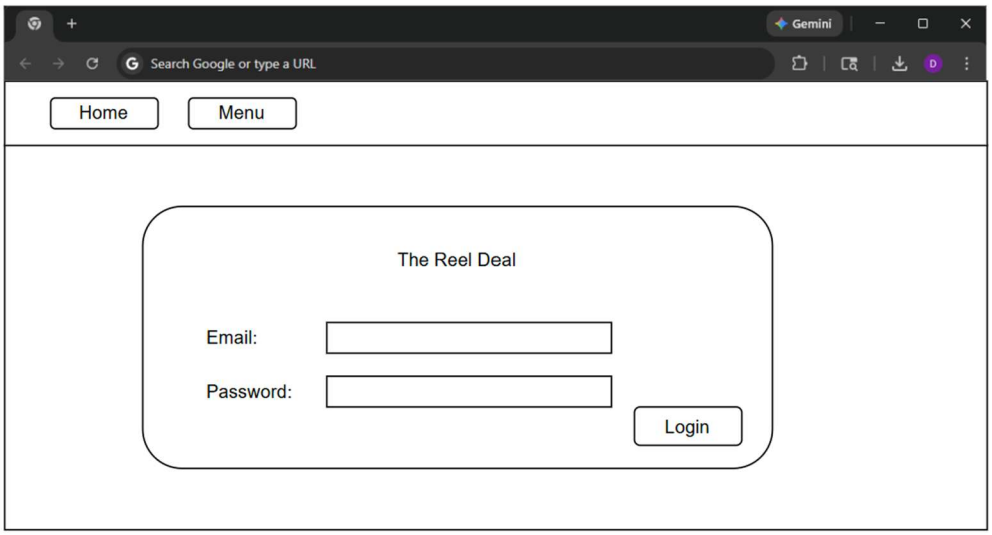
The following use cases are included:

- Login
- View Movies and Showtimes
- Select Seats and Confirm Booking
- Display Booking Confirmation
- Validate Ticket

**Use Case 1: Login**

**Preliminary Design:**
The login screen lets users sign in by entering their email and password. There are two input fields and a Login button. Once the user enters their information and clicks Login, the system verifies the credentials and moves them to the movie selection screen if the login is successful. If the information is wrong, an error message is displayed so the user can correct it.

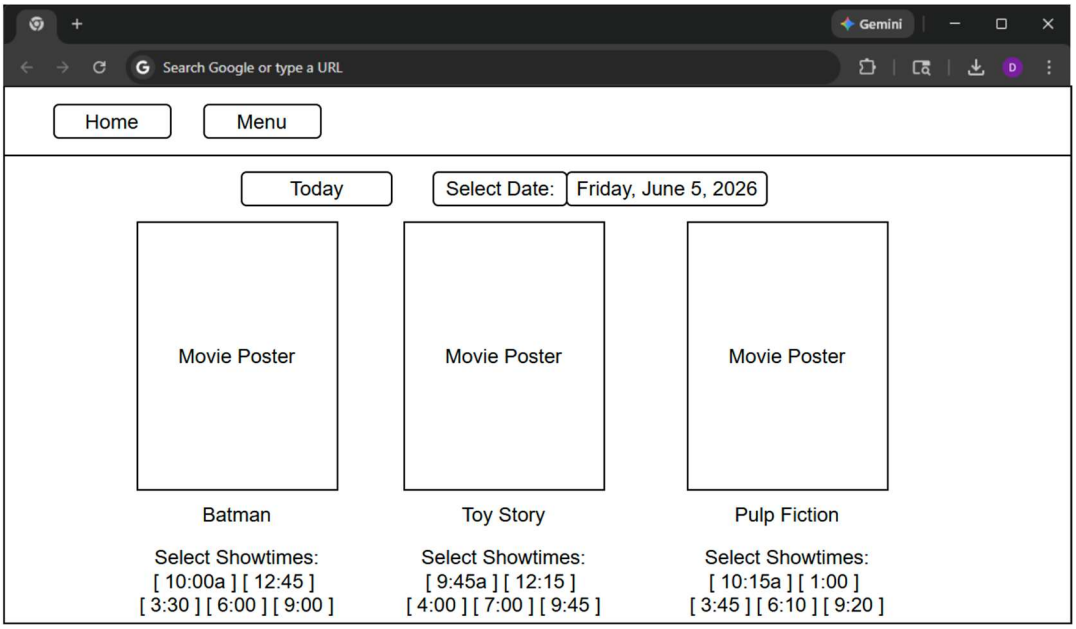
**Navigation Path:**
Home Screen > Login Screen > Movie Selection Screen



**Use Case 2: View Movies and Select Showtime**

**Preliminary Design:**
The movie selection screen lets users browse the movies that are currently available, along with their posters, titles, and showtimes. Each showtime is displayed as a clickable button. Once the user clicks a showtime, the system moves them to the seat selection screen for that movie so they can continue the booking process.

**Navigation Path:**
Login Screen > Movie Selection Screen > Seat Selection Screen

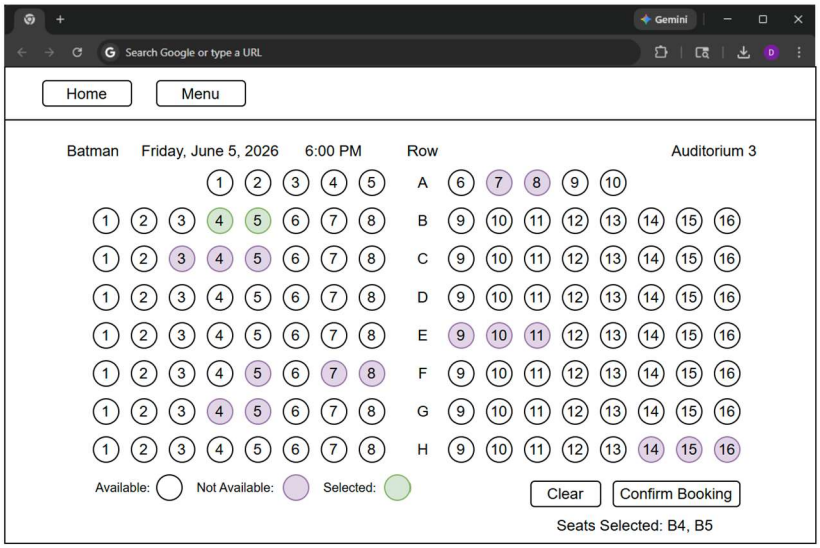



**Use Case 3: Select Seats and Confirm Booking**

**Preliminary Design:**
The seat selection screen lets the user see the seating layout for the selected movie and showtime. It clearly shows which seats are available and which ones are already taken. The user can click on available seats to select them and then click the Confirm Booking button. The system then verifies the seats and processes the booking.

**Navigation Path:**

Movie Selection Screen > Seat Selection Screen > Booking Confirmation Screen



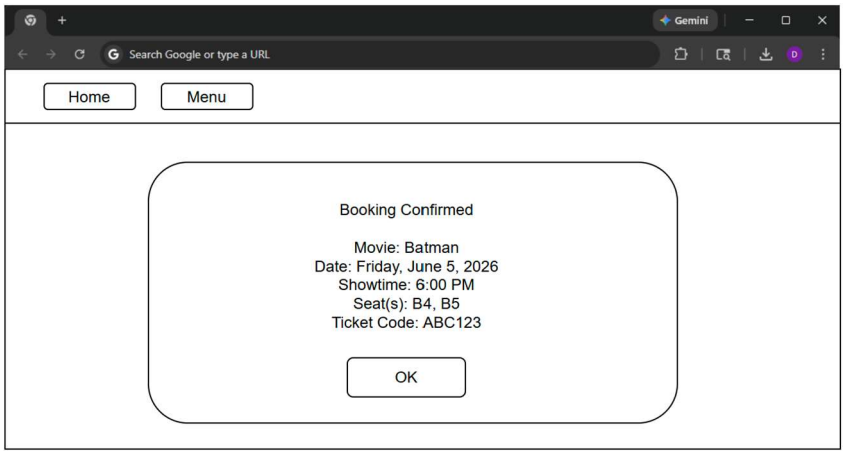

**Use Case 4: Display Booking Confirmation**

**Preliminary Design**

After confirming the booking, the system displays a confirmation screen showing the movie title, date, showtime, selected seats, and ticket code. This screen confirms that the booking was successful and provides the user with ticket information.

**Navigation Path**

Seat Selection Screen > Booking Confirmation Screen



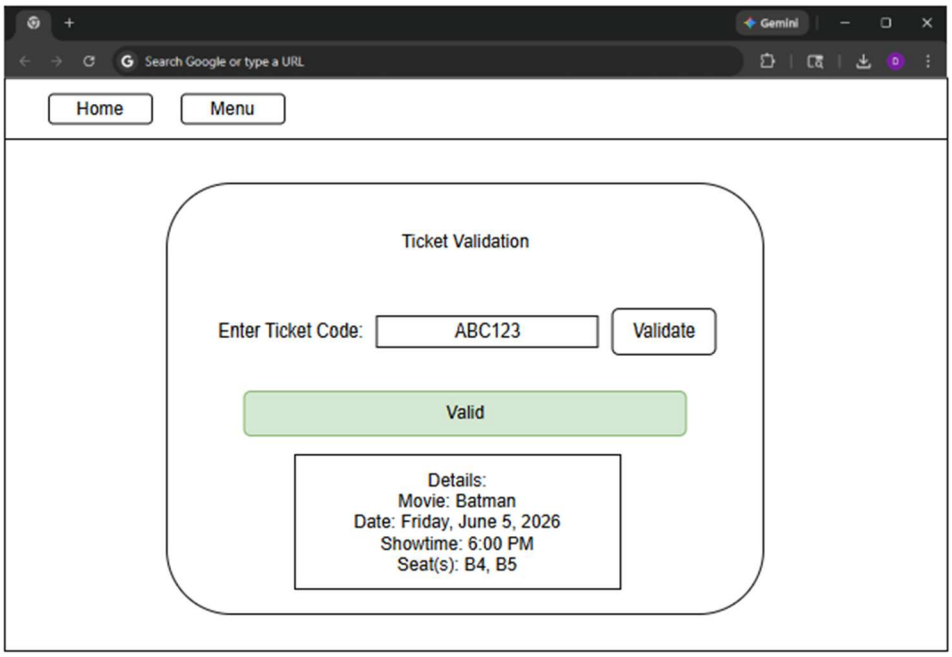
Use Case 5: Validate Ticket

**Preliminary Design:**

The ticket validation screen allows theater staff to scan or enter a ticket code. The system retrieves the booking record and checks whether the ticket is valid and unused. If valid, the system marks the ticket as used and allows entry. If invalid, the system displays an error message.

**Navigation Path:**

Staff Login > Ticket Validation Screen > Validation Result




## User Effort Estimation:

| Usage Scenario | Navigation | Clicks | Keystrokes |
|---|---|---|---|
| User Login | overview, enter email/password, login | 3 | <40 |
| View Movies and Showtimes | overview, select date, select showtime | 5 | 0 |
| Select Seats and Confirm | overview, select seat, confirm | 4 | 0 |
| Display Booking Confirmation | overview, okay | 1 | 0 |
| Validate Ticket | overview, enter ticket number, validate | 2 | <10 |

# Traceability Matrix

**System Requirements:**

| No. | Priority Weight | Description |
|---|---|---|
| REQ1 | 5 | Users can log in and log out of the system securely. |
| REQ2 | 5 | The system shows the list of available movies and their showtimes. |
| REQ3 | 5 | The system shows which seats are available for a selected showtime. |
| REQ4 | 5 | Customers can choose and reserve one or more seats. |
| REQ5 | 5 | The system prevents double booking of seats for the same showtime. |
| REQ6 | 4 | After a successful booking, the system generates a unique ticket code. |
| REQ7 | 4 | The system displays a confirmation screen with the booking details. |
| REQ8 | 4 | Theater staff can scan or enter a ticket code to validate it at the entrance. |
| REQ9 | 3 | Managers can add new movies and edit existing movies or showtimes. |

**Use Cases:**

| No. | Description |
|---|---|
| UC1 | User logs into the system. |
| UC2 | User views movies and selects a showtime. |
| UC3 | User views available seats for a selected showtime. |
| UC4 | User selects seats and confirms a booking. |
| UC5 | System generates ticket and displays booking confirmation. |
| UC6 | Theater staff validates a ticket at the entrance. |
| UC7 | Manager adds or edits movies and showtimes. |

**Traceability Matrix:**

| Req't | PW | UC1 | UC2 | UC3 | UC4 | UC5 | UC6 | UC7 |
|---|---|---|---|---|---|---|---|---|
| REQ1 | 5 | X | | | | | | |
| REQ2 | 5 | | X | | | | | |
| REQ3 | 5 | | | X | | | | |
| REQ4 | 5 | | | | X | | | |
| REQ5 | 5 | | | | X | | | |
| REQ6 | 4 | | | | | X | | |
| REQ7 | 4 | | | | | X | | |
| REQ8 | 4 | | | | | | X | |
| REQ9 | 3 | | | | | | | X |
| Max PW | | 5 | 5 | 5 | 5 | 4 | 4 | 3 |
| Total PW | | 5 | 5 | 5 | 10 | 8 | 4 | 3 |

# The Reel Deal Movie Booking System

## System Architecture and System Design

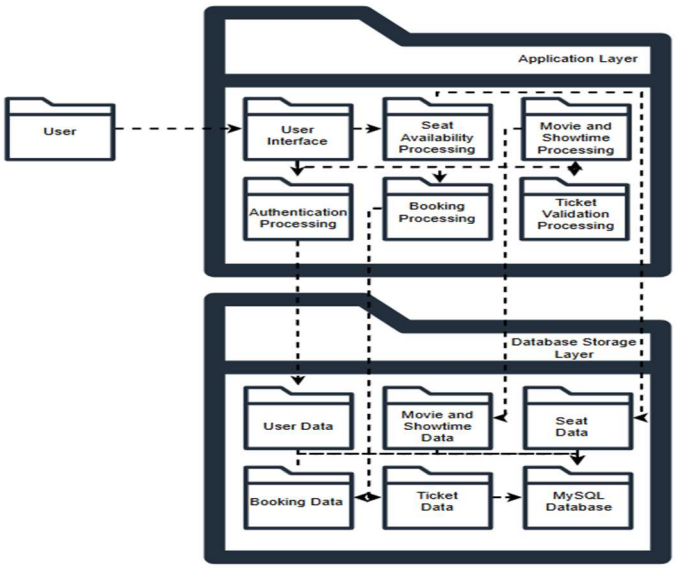
**Architectural Styles**

The Reel Deal movie booking system uses a client-server architectural style in which the client is the user's computer running a web browser and the server handles the application logic needed to support the system. Through the browser, users can log in, view movies and showtimes, check seat availability, and submit booking requests, while the server processes those requests by validating login credentials, retrieving movie and showtime data, checking whether seats are available, and completing reservations. The system also follows a layered architecture that separates the user interface, application logic, and database storage. The server communicates with a MySQL relational database that stores persistent data such as user accounts, seat availability, and booking records. This separation of responsibilities keeps the client simple and makes the system easier to understand, maintain, and expand in the future.

**Identifying Subsystems**

The UML package diagram for The Reel Deal system shows the main subsystems as the user interface subsystem, the application processing subsystem, and the database storage subsystem. The user interface subsystem contains the screens that users interact with, such as the login screen, movie selection screen, and seat selection screen, which allow users to enter information and view system results. The application processing subsystem contains the main logic of the system and is responsible for authenticating users, retrieving showtime information, checking seat availability, and processing seat reservations. The database storage subsystem stores the persistent data used by the system, including tables for users, seats, showtimes, and bookings. The application processing subsystem communicates with the database by sending queries to retrieve, insert, or update data as needed.

## UML Package Diagram:




**Mapping Subsystems to Hardware**

The Reel Deal system maps its subsystems across multiple machines in a client-server architecture. The client subsystem runs on the user's computer through a web browser, which displays the interface and allows users to select movies, view showtimes, and reserve seats. The application subsystem runs on a server computer that receives requests from the client and executes the Python application logic for functions such as authentication, seat availability checks, and booking management. The database subsystem runs on a database server using MySQL to store persistent data such as user accounts, seats, showtimes, and booking records. Communication between the client and the application server takes place through HTTP requests, while the application server communicates with the database server through SQL queries.

**Persistent Data Storage**

The Reel Deal system requires persistent data storage because important system data must remain available after the application is closed and reopened. Users need to be able to log in with existing accounts, view current seat availability, and access booking information that was created during earlier sessions. Persistent objects in the system include users, seats, showtimes, and booking records, and these are stored as tables in a relational database. The storage management strategy for the system is MySQL because it supports structured data, reliable long-term storage, and relationships between tables, which are necessary for connecting users, showtimes, seats, and reservations.

**Network Protocol**

The Reel Deal system uses HTTP as the network protocol for communication between the client and the application server. HTTP is the standard protocol used by web browsers and web servers, which makes it appropriate for a web-based movie booking system. When a user performs an action such as logging in, selecting a showtime, viewing available seats, or reserving a seat, the browser sends an HTTP request to the server. The server processes the request, interacts with the database when needed, and returns a response containing the requested information or confirmation of the action. HTTP was chosen because it is widely supported, easy to implement in web applications, and well suited for the request-and-response communication used in this system.

## Global Control Flow

**Execution Orders**

The Reel Deal system uses an event-driven control flow because it waits for user actions before processing requests. Users interact with the system through web page elements such as buttons, forms, and links, and each action triggers a request that is sent to the server. For example, when a user selects a showtime, views available seats, or confirms a reservation, the system processes that specific request and returns the result to the user interface. Because the system is event driven rather than strictly linear, users may perform actions in different orders depending on how they move through the system.

**Time Dependency**

The Reel Deal system does not use timers or real-time scheduling. Instead, it operates as an event-response system in which processing only occurs when a user interacts with the application, such as logging in, selecting a showtime, or reserving a seat. The system does not perform periodic background tasks and does not have strict real-time deadlines that must be met within a fixed time interval. Its responses are expected to occur within a reasonable time for normal user interaction, but it is not considered a real-time system.

**Concurrency**

The current prototype of The Reel Deal does not explicitly implement multithreading within the application code. However, the system is intended to support concurrent access by multiple users through the server environment, where separate user requests can be handled independently. This allows multiple customers to browse movies, view seat availability, and attempt reservations at the same time. In a full implementation, concurrency control would be

important to prevent conflicts such as two users attempting to reserve the same seat simultaneously.

### Hardware Requirements

The Reel Deal system depends on several hardware resources in order to run properly. Users must have access to a computer or mobile device capable of running a modern web browser so they can view the interface and interact with the system. The system also requires a server computer capable of running the Python application logic and a MySQL database for persistent storage. In addition, the system requires a stable network connection so the client can communicate with the server and sufficient storage to maintain user, showtime, seat, and booking data. Minimum hardware requirements include:

- Color display with minimum resolution of 1024 x 768 pixels
- Computer capable of running a modern web browser
- Minimum 1 GB RAM
- Minimum 1 GB available hard disk storage
- Network connection for communication between the client and server

These resources are necessary to support normal system operation and allow users to access the booking features reliably.

## User Interface Design and Implementation, Design of Tests

User Interface Design and Implementation

The final user interface for The Reel Deal was implemented as a Flask web application using HTML templates and CSS. The interface was designed to keep the booking process simple and easy to follow. Customers begin at a login page, then move to a movie and showtime page where movies are displayed with posters, descriptions, and clickable showtime buttons. After selecting a showtime, the customer is taken to a visual seat map.

The seat selection page uses a four-row seating layout with rows A-D and seats 1-8. Available seats are shown in white, selected seats turn gray before submission, and reserved seats are shown in purple. This makes the current seat status clear to the user before they confirm the

booking. Customers can select more than one seat in the same order, which supports group reservations.

After the customer submits a booking, the system displays a confirmation page with the movie title, showtime, auditorium, selected seats, ticket count, ticket status, and ticket code. This gives the customer clear proof of the completed reservation.

The staff interface uses role-based navigation. Staff users can validate ticket codes, view booking records, and manage movies and showtimes. The Manage Movies page allows staff to add a movie title, description, showtime, auditorium number, and poster image. When a new showtime is added, the system automatically creates a full A-D by 1-8 seating layout for that showtime.

The user interface was implemented using shared template styling through the base.html file. This keeps the layout consistent across the application and provides common navigation, messages, forms, tables, movie cards, poster images, seat maps, and confirmation boxes.

Design of Tests

Testing focused on confirming that the application works correctly across the customer workflow, staff workflow, and database operations. The tests were organized into unit testing and integration testing.

Unit testing focused on smaller pieces of application logic. These tests included checking the ticket code generation format, validating allowed poster file extensions, and confirming that the seating layout follows the expected A-D by 1-8 structure. These tests helped verify that important helper functions worked before they were used in the full application.

Integration testing focused on full workflows across the browser interface, Flask routes, and MySQL database. The customer workflow was tested by logging in as a customer, viewing movies and showtimes, selecting multiple seats, submitting a booking, and confirming that a ticket code was generated. The seat selection workflow was also tested by returning to the same showtime and verifying that the selected seats appeared as reserved.

The staff workflow was tested by logging in as a staff user, validating an active ticket code, confirming that the ticket status changed to used, and then attempting to validate the same ticket again. The system correctly identified the ticket as already used. The staff booking

dashboard was also tested to confirm that booking records displayed correctly from the database.

The staff movie management workflow was tested by adding a new movie, description, showtime, auditorium number, and poster image. After the movie was added, the customer account was used to confirm that the new movie appeared in the movie list and that the new showtime had an automatically generated seating layout.

The main test cases included:

1. Customer login
2. Staff login
3. Movie and showtime display
4. Multi-seat selection
5. Booking confirmation
6. Seat status update after booking
7. Ticket validation
8. Used-ticket prevention
9. Staff booking dashboard display
10. Staff movie and showtime creation
11. Automatic seat creation for new showtimes

Testing confirmed that the system supports the main project requirements, including role-based access, movie browsing, seat reservation, double-booking prevention, ticket generation, ticket validation, booking record storage, and staff movie management.

# Project Plan

| Week | Task | Status |
|---|---|---|
| 1-2 | Set up development environment, design database schema, connect front end to back end | Completed |
| 3-4 | Implement login and registration features | Completed |
| 5-7 | Develop manager features for movies, showtimes, and seating layouts | Completed |
| 8 | Midterm testing and demo recording | Completed |
| 9-11 | Develop customer booking features and seat reservation | Completed |
| 12-14 | Write test cases, fix bugs, improve usability | Completed |
| 15 | Final testing, documentation, and demo | Completed |

By the end of the semester, The Reel Deal was completed as a working Flask and MySQL movie ticket booking system. The final version includes customer and staff login, movie and showtime browsing, movie poster display, multi-seat selection, automatic seat reservation, ticket code generation, ticket validation, and a staff booking dashboard. The system also includes staff management functionality that allows staff users to add new movies, descriptions, showtimes, auditorium numbers, and poster images.

The final implementation expanded the project beyond the midpoint prototype. At the midterm, the system was mainly focused on database planning, early authentication testing, and basic booking logic. The completed version now includes a full web interface, role-based navigation, automatic creation of A-D by 1-8 seating layouts for new showtimes, and database updates that prevent reserved seats from being booked again. Final testing focused on confirming that customers can complete the booking process, staff can validate ticket codes, and new movies and showtimes can be added successfully.

# References

Tilley, S., & Rosenblatt, H. J. (2017). Systems analysis and design (11th ed.). Cengage Learning.